

sklearn wrapper vs native API

사용자

머신러닝 프로그래밍 할때 sklearn wrapper 모듈, python wrapper 모듈 방식이 있는데 왜 이렇게 나눠져 있고 언제, 어떻게 사용하는지 자세히 비교해 줘.

Claude

머신러닝 라이브러리들(XGBoost, LightGBM, CatBoost 등)이 sklearn wrapper API와 native(python) API 두 가지 방식을 함께 제공하는 이유와 차이를 자세히 설명드리겠습니다. ADP 실기가 파이썬 기반이시니 실전에서 어떤 걸 쓰면 좋을지도 함께 짚어드릴게요.

왜 두 가지 방식이 존재하는가

XGBoost, LightGBM 같은 라이브러리들은 원래 각자의 네이티브 C++ 엔진을 파이썬에서 다루기 위한 고유 인터페이스(native API)를 먼저 만들었습니다. 이 인터페이스는 라이브러리 고유의 최적화 기능(예: 조기 종료, 커스텀 손실함수, GPU 옵션 등)을 세밀하게 제어할 수 있도록 설계되어 있습니다.

문제는 scikit-learn 생태계가 워낙 넓다는 점입니다. Pipeline, GridSearchCV, cross_val_score, VotingClassifier 등 수많은 도구들이 fit(), predict(), predict_proba(), get_params(), set_params() 같은 정해진 인터페이스(estimator API)를 따르는 객체만 받아들이도록 만들어져 있습니다.

그래서 각 라이브러리는 자신의 네이티브 엔진을 감싸서 sklearn 규격에 맞춘 wrapper 클래스(예: XGBClassifier, LGBMRegressor)를 추가로 제공하게 된 것입니다. 즉:

- Native API: 라이브러리 고유의 데이터 구조와 함수 (예: xgb.DMatrix, xgb.train())
- sklearn Wrapper API: sklearn의 표준 규격에 맞춘 클래스 (예: xgb.XGBClassifier().fit())

상세 비교

구분	sklearn Wrapper API	Native(Python) API
예시 (XGBoost)	XGBClassifier, XGBRegressor	xgb.train(), xgb.DMatrix
예시 (LightGBM)	LGBMClassifier, LGBMRegressor	lgb.train(), lgb.Dataset
데이터 입력	numpy array, DataFrame 그대로 사용	전용 데이터 구조로 변환 필요 (DMatrix, Dataset)
학습 방식	.fit(X, y)	train(params, dtrain, ...)
예측 방식	.predict(X), .predict_proba(X)	.predict(dtest) (확률/로짓 등 반환 형태가 다름)
파라미터 지정	생성자 인자로 지정 (XGBClassifier(max_depth=3, ...))	params 딕셔너리로 전달
Pipeline 연동	가능 (Pipeline, ColumnTransformer에 바로 사용)	불가능 (직접 감싸야 함)
GridSearchCV / RandomizedSearchCV	바로 사용 가능	직접 CV 루프를 짜야 함
조기 종료(early stopping)	가능하지만 버전별로 인자명이 다소 제한적	세밀한 제어 가능 (early_stopping_rounds, 커스텀 평가 함수 등)
커스텀 목적함수/평가함수	제한적 지원	완전한 지원
GPU/분산 학습 세부 옵션	지원하나 일부 제약	모든 저수준 옵션 접근 가능
학습 속도/메모리	거의 동일 (내부적으로 native 엔진 호출)	거의 동일
모델 저장/불러오기	joblib, pickle 등 sklearn 관례 그대로 사용	자체 포맷(save_model) 권장
코드 스타일	scikit-learn 스타일 (다른 모델과 통일감)	라이브러리 고유 스타일

언제, 어떻게 사용하는가

sklearn Wrapper를 쓰는 경우

- 여러 모델(LogisticRegression, RandomForestClassifier, XGBClassifier 등)을 같은 방식으로 비교/실험할 때
- Pipeline으로 전처리+모델을 하나로 묶고 싶을 때
- GridSearchCV, RandomizedSearchCV, cross_val_score로 하이퍼파라미터 튜닝이나 교차검증을 자동화하고 싶을 때
- VotingClassifier, StackingClassifier 등 앙상블 조합에 넣고 싶을 때
- 빠르게 프로토타입을 만들고 코드 일관성을 유지하고 싶을 때 (실무에서 대부분 이 방식을 기본으로 씀)

```
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV

model = XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.1)
grid = GridSearchCV(model, param_grid={"max_depth":[3,4,5]}, cv=5)
grid.fit(X_train, y_train)
```

Native API를 쓰는 경우

- 대용량 데이터에서 메모리 효율(DMatrix/Dataset의 내부 압축·캐싱)을 극대화하고 싶을 때
- 커스텀 손실함수, 커스텀 평가지표를 직접 정의해서 학습시켜야 할 때
- 조기 종료를 학습 로그와 함께 세밀하게 제어하고 싶을 때(검증셋 여러 개 모니터링 등)
- 캐글 등 성능을 극한까지 끌어올려야 하는 상황에서 라이브러리 고유 기능(예: LightGBM의 feature_fraction_bynode, XGBoost의 DMatrix 캐시)을 모두 활용하고 싶을 때
- 프레임워크(sklearn) 오버헤드 없이 저수준 제어가 필요할 때

```
import xgboost as xgb

dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test)
params = {"max_depth":4, "eta":0.1, "objective":"binary:logistic"}
bst = xgb.train(params, dtrain, num_boost_round=200,
```

```
evals=[(dtrain,"train")], early_stopping_rounds=10)
preds = bst.predict(dtest)
```

실전 팁 (ADP 실기 관점)

ADP 실기처럼 시간 제한이 있고 다양한 모델을 빠르게 비교·튜닝해야 하는 상황에서는, 특별히 커스텀 손실함수가 필요하지 않다면 sklearn wrapper 방식이 압도적으로 편리합니다. Pipeline + GridSearchCV 조합으로 전처리부터 모델 선택, 튜닝까지 한 번에 처리할 수 있고, 다른 sklearn 모델들과 코드 스타일이 통일되어 실수도 줄어듭니다. Native API는 세밀한 제어가 필요한 특수한 상황(대회 수준의 성능 튜닝, 커스텀 목적함수)에서 선택적으로 쓰는 것이 일반적입니다.